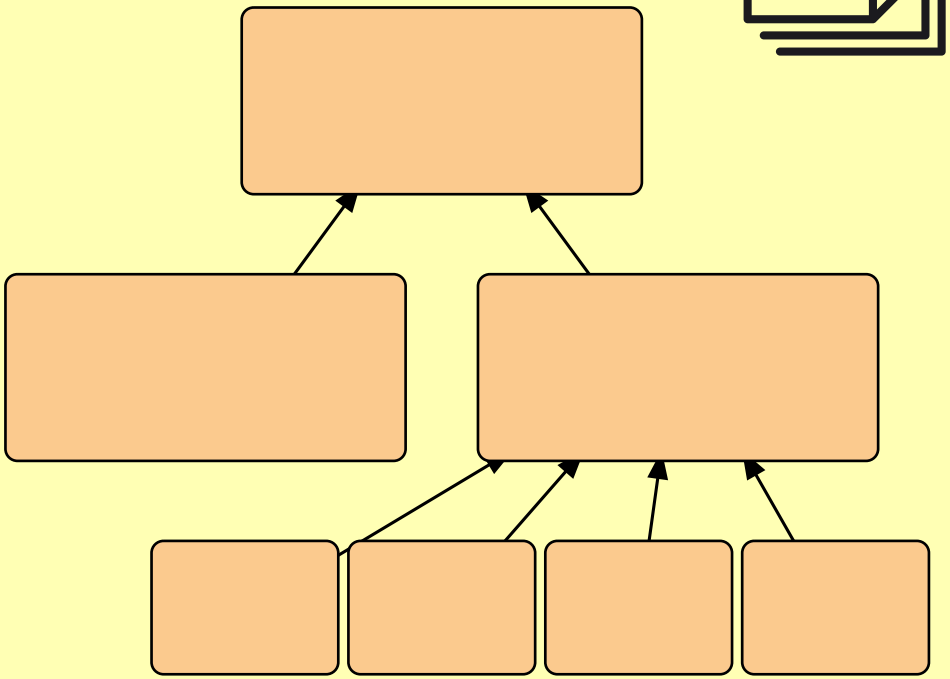
Starter: Parts of an OS

1. Match the keywords to the boxes in the diagram:
   - Translators
   - Application Software
   - Libraries
   - Software
   - Operating Systems
   - Utility programs
   - System software
2. Describe two roles of an operating system



Prework Quiz

Define the term programming paradigm

Prework Quiz

Define the term programming paradigm

A way or style of programming

Prework Quiz

Which of the following are programming paradigms

`Imperative, Object-oriented, Instructional, Fundamental, Functional, Procedural, Artificial Intelligence, Declarative`

Prework Quiz

Which of the following are programming paradigms

`Imperative, Object-oriented, Instructional, Fundamental, Functional, Procedural, Artificial Intelligence, Declarative`

`Imperative, Object Oriented, Functional, Procedural, Declarative`

Prework Quiz

What is the difference between a high-level and a low-level programming language?

Prework Quiz

What is the difference between a high-level and a low-level programming language?

One instruction in high-level maps to many in low-level. Low-level is close to the hardware, whereas high-level is abstracted from it.

Prework Quiz

Give **one** advantage of low-level programming languages

Prework Quiz

Give **one** advantage of low-level programming languages

Faster, Less resource used, closer to hardware

Prework Quiz

Give **one** advantage of high-level programming languages

Prework Quiz

Give **one** advantage of high-level programming languages

More achieved in less code, Easier for programmers to understand, Easier to debug, Processor-independent

Which types of language would you use to design:

1. Microsoft Word
2. Database Development
3. Videogame
4. Computer Virus

Your **justification** is very important here. Make sure to include your reasoning!

Translators

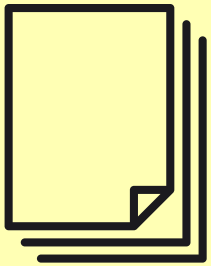
We need to understand the concept behind three different types of translator. You've already encountered one!

1. Assemblers
2. Interpreters
3. Compilers

Assemblers

Assemblers turn **assembly code** into **machine code**.

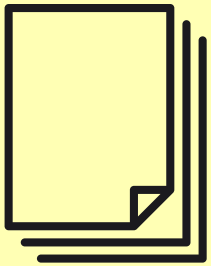
Assemblers



Assemblers turn **assembly code** into **machine code**.

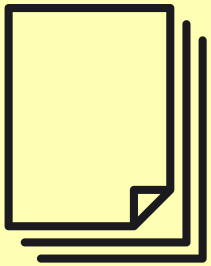
This is a **one-to-one** mapping, like looking things up in the dictionary.

Interpreters



These translate **single lines** of high-level code into **machine code**.

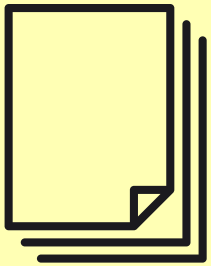
Interpreters



These translate **single lines** of high-level code into **machine code**.

These lines are then **run**.

Interpreters

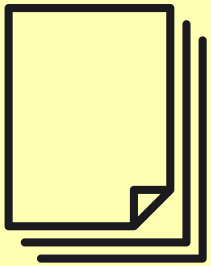


These translate **single lines** of high-level code into **machine code**.

These lines are then **run**.

They then translate the next line, and so on.

Interpreters



These translate **single lines** of high-level code into **machine code**.

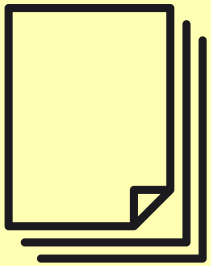
These lines are then **run**.

They then translate the next line, and so on.

While these may be a one-to-one mapping, they may not be.

These lines of machine code are not saved.

Compilers

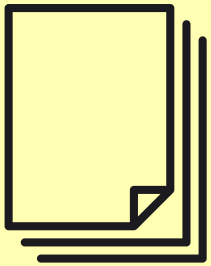


These translate the **entire program** into **machine code**.

They do **not** run the program.

They translate **behaviours**, rather than specific lines.

Compilers v Interpreters



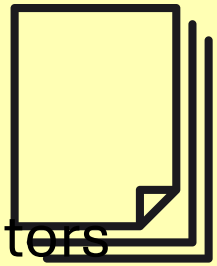
Interpreters:

- Do not produce object code
- Interpret (and execute) code line by line
- Can start straight away
- Only need to interpret the line being run
- Need time to interpret as the program runs … every time the program runs
- Need original source code and programming environment every time

Compilers

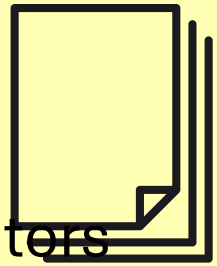
- Do not execute the program
- Need a setup time to complete compilation
- Creates an object file that can be run without original source code and programming environment
- Object file (executable) will run quicker than interpreting
- Can hide original source code

Bytecode



Some compiled programming languages (e.g. C#, Java) use translators which don’t immediately translate to machine code.

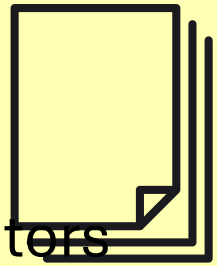
Bytecode



Some compiled programming languages (e.g. C#, Java) use translators which don’t immediately translate to machine code.

Instead they translate to an intermediate language called bytecode.

Bytecode



Some compiled programming languages (e.g. C#, Java) use translators which don't immediately translate to machine code.

Instead they translate to an intermediate language called bytecode.

When the program is run, this bytecode is typically interpreted*.

*in practice, C# and Java bytecode is compiled and run using a just-in-time (JIT) compiler, whereas Python bytecode is often interpreted by its higher-level virtual machine

Why Bytecode?

This method is a compromise between **compilation** and **interpretation**.

Why Bytecode?

This method is a compromise between **compilation** and **interpretation**.

Bytecode has the advantage of being relatively **easy to interpret** for the hardware.

Why Bytecode?

This method is a compromise between **compilation** and **interpretation**.

Bytecode has the advantage of being relatively **easy to interpret** for the hardware.

But it is also **independent** of any particular processor instruction set – **portable**.

Which translator should be used:

1. Writing and debugging a complex program in a test environment
2. Writing a printer device driver
3. Finished version of a computer game
4. Complex calculator used by research scientists
5. Word processing program
6. Computer Virus

True or False?

“An interpreter converts code (one line at a time) from the source language into machine code, then executes it”

True or False?

“An interpreter converts code (one line at a time) from the source language into machine code, then executes it”

False, kind of.

True or False?

“An interpreter converts code (one line at a time) from the source language into machine code, then executes it”

False, it interprets **behaviour**, but not the code directly.

True or False?

“An interpreter converts code (one line at a time) from the source language into machine code, then executes it”

False, it interprets **behaviour**, but not the code directly.

An interpreter generally uses one of the following strategies for program execution:

- Parse the source code and perform its behaviour directly;
- Translate source code into some efficient intermediate representation and immediately execute this;
- Explicitly execute stored precompiled code made by a compiler which is part of the interpreter system.

Multi-dimensional Arrays

Multi-dimensional arrays

1. What are the differences between normal arrays and 2D arrays?

2. Which index comes first, row or column?

Multi-dimensional arrays

1. What are the differences between normal arrays and 2D arrays?

An extra dimension

An additional index to address elements

2. Which index comes first, row or column?

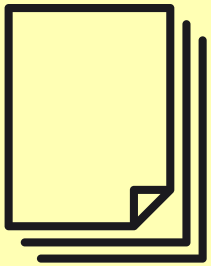
Neither! (very common misconception)

Example Array!

Creating 2D Arrays

```
int[,] grid = new int[4,4];
grid[2, 1] = 5
```

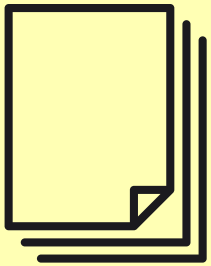
Creating 2D Arrays



```
int[,] arr = new int[4, 4];
for(int first = 0; first < 4; first++)
{
    for(int second = 0; second < 4; second++)
    {
        arr[first, second] = first + second;
    }
}
```

What does this produce?

Creating 2D Arrays



```
int[,] T = new int[4, 4];
for(int first = 0; first < 4; first++)
{
    for(int second = 0; second < 4; second++)
    {
        T[first, second] = first * second;
    }
}
```

What does this produce?

You Try

Try in C# filling a 5x5 array with random numbers between 1 and 9 and display as a table

Extension: Make the random numbers between 1 and 999 and make the table line up nicely

Extension: Modify your program to find the range, mean and mode of all the numbers in the grid

Practice Time!

On BPCompSci

W121 – C# 2D Arrays